Informe – Actividad Grupal Clase 9

Construcción completa del perceptrón con descenso del gradiente · Inteligencia Artificial 2026-02

Campo	Respuesta
Grupo N.°	**05**
Fecha	19 de octubre de 2026

Integrante	Rol
Monica Parra	Coordinador(a)
Brandon Cortes	Programador(a) inicial
Miguel Ramirez	Verificador(a)
Andres Mejia Suaza	Relator(a)

1. Diagnóstico (Misión 0)

¿La neurona llega a cero errores? ¿Qué pasa con los errores de una época a otra?

Respuesta: No, para nada. La neurona de la clase 8 nunca llega a cero errores. Arranca con 50 errores de los 120 lotes (más del 41% de fallo) y de ahí se queda bailando y oscilando entre 29 y 35 errores por época, terminando en la época 30 estancada en 31 errores (apenas un 77.5% de exactitud). Esto pasa porque con la regla de Hebb y esa función escalón tan rígida, cuando los datos se cruzan o no se pueden separar de forma perfecta con una sola línea recta, la neurona nunca converge y se queda dando tumbos de un lado a otro.

¿Qué ocurre con el tamaño de w1 y w2? ¿Por qué?

Respuesta: Los pesos se crecen una barbaridad y quedan totalmente descuadrados. Empezaron en 0 y al final de las 30 épocas w1 terminó en 33.86 mientras que w2 se disparó hasta -139.20. La razón es que las dos variables vienen en escalas muy distintas: la lluvia se mueve entre 40 y 260 mm, mientras que la temperatura va de 20 a 36 °C. Al multiplicar los errores por números de lluvia tan grandes y sin normalizar nada, los pesos se van inflando época tras época sin ningún control.

¿Qué tan segura está la neurona para el lote de 150 mm y 26 °C? ¿Le sirve a la cooperativa?

Respuesta: No podemos saber qué tan segura está, porque esa neurona bota un 0 o un 1 a machete (todo o nada). Justo en ese lote de 150 mm y 26 °C estamos en plena zona de duda (en la frontera), y que el modelo diga '1' sin decirnos si está al 51% o al 99% no

le sirve para nada a la cooperativa de Cartago. Un agricultor que va a meter plata en semillas, abono y jornales necesita saber el nivel de riesgo o la probabilidad real para decidir si se arriesga a sembrar o no.

¿Qué pasaría con 1 millón de lotes?

Respuesta: Se nos quedaría pegado el computador. El código viejo usa ciclos for recorriendo lote por lote uno a uno en Python puro. Con 1 millón de registros eso se demoraría una eternidad (minutos u horas) y se comería toda la memoria. Para eso es que se necesita vectorizar con NumPy, para que las operaciones se hagan de un solo golpe matricial aprovechando el procesador en milisegundos.

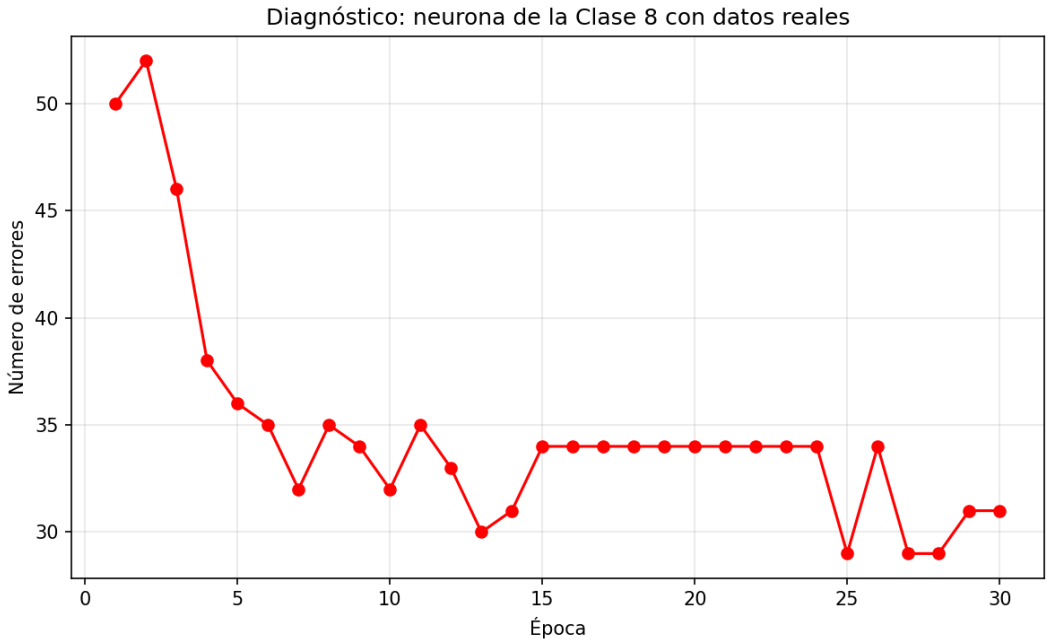


Figura 1: Diagnóstico de la Neurona de la Clase 8 (Errores por época y frontera oscilante).

1. Vectorización con NumPy: Quitar esos ciclos for tan lentos y meter operaciones matriciales (np.dot) para procesar todos los lotes de un solo golpe.

1. 2. Función de activación Sigmoide: Cambiar el escalón rígido por una curva suave que nos entregue probabilidades reales entre 0 y 1 en vez de un sí o no a secas.
2. 3. Entropía cruzada y Descenso del Gradiente: Medir el error con una fórmula matemática continua y calcular los gradientes para que los pesos vayan bajando hacia el mínimo error y no al tanteo.

3. 4. Estandarización de entradas (Z-Score): Nivelar la lluvia y la temperatura a media 0 y desviación 1 para que las diferencias de escala no dañen los cálculos ni disparen los pesos.
4.

2. Bitácora de Retos

RETO 1 – Vectorización

Aspecto	Registro del grupo
¿Qué faltaba?	Calcular el potencial z de todos los lotes al tiempo sin tener que hacer un ciclo for lote por lote.
¿Cómo lo descubrieron?	Revisamos el TODO del código y vimos que si multiplicamos la matriz X por el vector w usando np.dot(X, w) + b, NumPy nos calcula de una vez todo el vector z con los 120 resultados.
Piloto en este reto	Brandon Cortes
Código de la solución	def suma_ponderada(X, w, b): return np.dot(X, w) + b
Evidencia (salida)	[OK] RETO 1 (vectorización) superado (z tiene dimensión (120,) y dio los valores exactos para la compuerta AND de prueba: [-0.7, -0.2, -0.2, 0.3])

RETO 2 – Activación con probabilidad

Aspecto	Registro del grupo
¿Qué faltaba?	Una función suave que convierta cualquier número z en una probabilidad real entre 0 y 1.
¿Cómo lo descubrieron?	Buscamos la fórmula de la sigmoide que nos explicó el profe: 1 / (1 + e^-z) y la programamos directo en NumPy usando np.exp(-z).
Piloto en este reto	Monica Parra
Código de la solución	def sigmoide(z): return 1.0 / (1.0 + np.exp(-z))
Evidencia (salida)	[OK] RETO 2 (función sigmoide) superado (sigmoide([-6.0, 0.0, 6.0]) = [0.0025, 0.5, 0.9975], dando 0.5 exacto en el centro)

RETO 3 – Medir el error

Aspecto	Registro del grupo
¿Qué faltaba?	Una fórmula para medir qué tan bien o mal estaba prediciendo la neurona usando probabilidades.
¿Cómo lo descubrieron?	Aplicamos la fórmula de Entropía Cruzada Binaria (BCE) y le pusimos el np.clip que sugería la plantilla para no tener problemas de logaritmo de cero si daba 0 o 1.
Piloto en este reto	Miguel Ramirez
Código de la solución	def calcular_perdida(y_real, y_prob): epsilon = 1e-9 y_prob = np.clip(y_prob, epsilon, 1 - epsilon) return -np.mean(y_real * np.log(y_prob) + (1.0 - y_real) * np.log(1.0 - y_prob))
Evidencia (salida)	[OK] RETO 3 (función de pérdida) superado (Pérdida inicial cuando duda con 0.5 = 0.6931 | Pérdida con acierto seguro de 0.9 = 0.1054)

RETO 4A – Dirección del aprendizaje

Aspecto	Registro del grupo
¿Qué faltaba?	Saber hacia qué lado mover los pesos y el sesgo para ir bajando el error en cada paso.
¿Cómo lo descubrieron?	Sacamos la derivada del error: restamos (y_prob - y_real), multiplicamos por la transpuesta de X con np.dot(X.T, error) / n para los pesos, y promediamos el error para el sesgo b.
Piloto en este reto	Andres Mejia Suaza
Código de la solución	def calcular_gradientes(X, y_real, y_prob): n = len(y_real) error = y_prob - y_real gradiente_w = np.dot(X.T, error) / n gradiente_b = np.mean(error) return gradiente_w, gradiente_b
Evidencia (salida)	[OK] RETO 4A (gradientes) superado (gradiente_w = [-0.25, 0.25] y gradiente_b = 0.0 para la prueba de control)

RETO 4B – Paso de aprendizaje

Aspecto	Registro del grupo
¿Qué faltaba?	La regla para ir actualizando los pesos en cada época durante el entrenamiento.
¿Cómo lo descubrieron?	Nos acordamos de que para ir 'cuesta abajo' hacia el menor error toca restar el gradiente multiplicado por la tasa de aprendizaje: w = w - lr * grad_w y b = b - lr * grad_b.
Piloto en este reto	Brandon Cortes

Código de la solución	# Actualización en cada época:
	w = w - tasa_aprendizaje * gradiente_w
	b = b - tasa_aprendizaje * gradiente_b

Evidencia (salida)	[OK] RETO 4B (actualización de pesos) superado (Época 1 | Pérdida: 0.6931 | w=[0.028, -0.025] b=0.004 moviéndose en la dirección correcta)

RETO 5 – Preparación de las entradas

Aspecto	Registro del grupo
¿Qué faltaba?	Estandarizar las entradas para que la lluvia (que llega a 260) y la temperatura (que va a 36) queden en la misma escala.
¿Cómo lo descubrieron?	Al correrlo sin esto nos salió la advertencia RuntimeWarning: overflow in exp porque la exponencial se reventaba. Le sacamos la media y desviación a X con NumPy y normalizamos.
Piloto en este reto	Monica Parra
Código de la solución	media = X.mean(axis=0) desviacion = X.std(axis=0) X_esc = (X - media) / desviacion
Evidencia (salida)	[OK] RETO 5 (preparación de las entradas) superado (Media ~ 0, desviación ~ 1, sin advertencias de overflow, pérdida final 0.2281 y exactitud subió al 90%)

RETO 6 – Frontera de decisión

Aspecto	Registro del grupo
¿Qué faltaba?	Trazar la recta divisoria donde la neurona duda totalmente (probabilidad 0.5, o sea w1*x1 + w2*x2 + b = 0).
¿Cómo lo descubrieron?	Despejamos x2 de la ecuación: pasamos todo al otro lado y dividimos entre w2, dando x2 = -(w1*x1 + b) / w2 en la escala normalizada, y luego la devolvimos a la escala real para dibujarla.
Piloto en este reto	Miguel Ramirez
Código de la solución	x1_original = np.linspace(X[:, 0].min(), X[:, 0].max(), 100) x1_esc = (x1_original - media[0]) / desviacion[0] x2_esc = -(w[0] * x1_esc + b) / w[1] x2_original = x2_esc * desviacion[1] + media[1]
Evidencia (salida)	[OK] RETO 6 (frontera de decisión) superado (Gráfico guardado en frontera_decision.png separando claramente los lotes exitosos de los no exitosos)

RETO 7 – Predecir lotes nuevos

Aspecto	Registro del grupo
¿Qué faltaba?	Evaluar los 3 lotes nuevos que nos pidió el problema usando el modelo ya entrenado.
¿Cómo lo descubrieron?	Tuvimos en cuenta que a los lotes nuevos había que restarles la misma media y dividirlos entre la misma desviación de los 120 lotes del entrenamiento antes de pasarlos por la sigmoide.
Piloto en este reto	Andres Mejia Suaza
Código de la solución	lotes_nuevos_esc = (lotes_nuevos - media) / desviacion for (lluvia, temp), prob in zip(lotes_nuevos, predecir_probabilidad(lotes_nuevos_esc, w, b)): decision = 'EXITOSA' if prob >= 0.5 else 'NO EXITOSA'
Evidencia (salida)	[OK] RETO 7 (predicción de lotes nuevos) superado (220mm/23.5°C -> 99.48% EXITOSA | 60mm/30°C -> 0.11% NO EXITOSA | 150mm/26°C -> 55.65% EXITOSA)

3. Resultados

Resultado	Antes del reto 5	Después del reto 5
Pérdida inicial	5.2830	0.6931
Pérdida final	5.2830 (se quedó trabada)	0.2281 (bajó derechito)
Exactitud final	74.17%	90.00%
w1 / w2 / b	w1=3.3155, w2=-25.2997, b=0.1440	w1=2.6217, w2=-2.3658, b=0.3152
¿Apareció alguna advertencia?	Sí (RuntimeWarning: overflow in exp)	Ninguna (corrió limpiecito y sin errores)

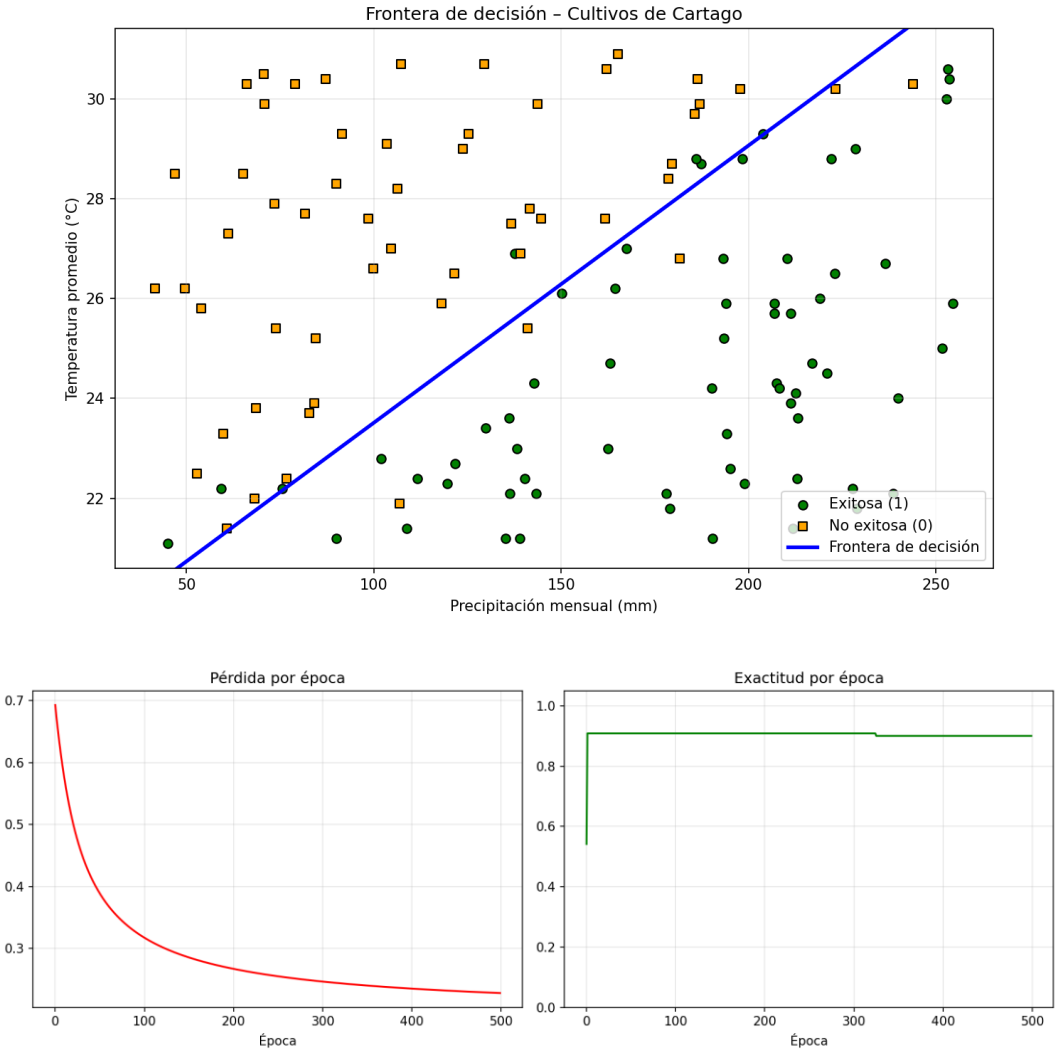


Figura 2: Curva de Aprendizaje (Pérdida y Exactitud) y Frontera de Decisión Resultante.

Imágenes: pegar `curva_aprendizaje.png` y `frontera_decision.png`.

Lote nuevo	Probabilidad	Decisión
220 mm / 23.5 °C	99.48%	EXITOSA (1)
60 mm / 30 °C	0.11%	NO EXITOSA (0)
150 mm / 26 °C	55.65%	EXITOSA (1)

Respuesta: Para cualquier campesino o agricultor acá en la zona de Cartago, esto tiene toda la lógica del mundo:

• w1 = +2.6217 (Signo positivo): Significa que entre más llueva (buena precipitación), mucho mejor le va al cultivo y más sube la probabilidad de tener una cosecha exitosa. El agua suma a favor.

• w2 = -2.3658 (Signo negativo): Significa que las temperaturas muy altas castigan el cultivo. En Cartago cuando hace calores bravos se secan las matas o se estresan por temperatura, lo que tumba la probabilidad de éxito de la cosecha.

• Magnitudes relativas: Como ambos números están casi parejos en valor absoluto (2.62 contra 2.37), nos muestra que el clima de Cartago depende tanto de que no falte agua como de que el calor no se pase de la raya; ambas cosas pesan casi lo mismo.

4. Interpretación

Respuesta: Cambió del cielo a la tierra. Antes de estandarizar, la lluvia (que son números de cientos) opacaba por completo a la temperatura (que son números de decenas). Eso hacía que el gradiente fuera larguísimo y deforme, y la función exponencial exp(-z) se reventara dando el error de overflow. El modelo se quedaba trabado en una pérdida altísima de 5.2830. Cuando aplicamos Z-score en el reto 5, ambas variables quedaron niveladas con media 0 y varianza 1; el descenso del gradiente bajó directico sin dar tumbos, la pérdida cayó a 0.2281, la exactitud subió al 90% y no volvió a salir ninguna advertencia.

Respuesta:

¿Qué cambió con el reto 5 y por qué?

Respuesta:

5. Experimento

Experimento elegido (A, B o C)	Experimento A: Cambiar la tasa de aprendizaje (learning rate, lr)
Qué se cambió	Probamos variando la tasa de aprendizaje con tres valores distintos: lr = 0.001, lr = 0.1 (el que venía) y lr = 1.0, dejando fijas las 500 épocas.
Resultado	• Con lr = 0.001: El aprendizaje se quedó dormido. Apenas bajó a pérdida de 0.6289 y los pesos casi ni se movieron (w=[0.485, -0.414], b=0.065). Le harían falta miles de épocas más para llegar al mínimo. • Con lr = 0.1: Es el punto perfecto. La curva de pérdida baja suave y parejita, llegando a 0.2281 y 90% de exactitud sin saltos raros. • Con lr = 1.0: Dio una pérdida bajita de 0.2050, pero en las primeras épocas los pasos son demasiado bruscos y agresivos, lo que con datos más ruidosos podría hacer que el modelo se descarrile o diverja.
Conclusión	Nos quedamos con lr = 0.1 porque es la tasa más equilibrada: aprende rápido, no da brincos raros y converge con total estabilidad.

Lo que no nos imaginábamos que faltaba: Sinceramente no nos imaginábamos que el escalado de los datos (el Z-Score) fuera tan definitivo. Pensábamos que la neurona solita iba a compensar la diferencia entre milímetros y grados ajustando los pesos a punta de gradiente. Al ver que sin normalizar salía advertencia de desbordamiento en NumPy y la pérdida no bajaba de 5.28, nos dimos cuenta de que si las entradas no están en la misma escala, las matemáticas del gradiente se enloquecen.

6. Hipótesis vs Realidad

Lo que creíamos que faltaba	¿Era necesario? ¿Por qué?
1. Vectorización con matrices en NumPy	Totalmente necesaria. Sin eso el código sería lentísimo y no podríamos procesar los lotes de un solo viaje ni calcular los gradientes por lote.
2. Función de activación Sigmoide	Fundamental. Nos cambió ese 0 y 1 secos por una probabilidad real entre 0% y 100% que sí le sirve a la gente de la cooperativa.
3. Entropía cruzada y Gradientes analíticos	Indispensable. Le dio dirección matemática al modelo para ir bajando el error con certeza y no a ciegas como hacía la regla de Hebb.
4. Estandarización de entradas (Z-Score)	Clave total. Sin esto el modelo ni siquiera entrenaba bien y se desbordaban las exponenciales en la computadora.

1. Pasar de la neurona vieja a este perceptrón completo con sigmoide y entropía cruzada es un salto gigante: dejamos de adivinar con una regla a machete y pasamos a un modelo matemático que calcula probabilidades reales de éxito para los cultivos de Cartago.

2. La estandarización de las variables no es un simple detalle cosmético: es obligatoria para que el gradiente no se estrelle ni se desborden las exponenciales cuando juntamos variables con rangos tan diferentes como lluvia y temperatura.

3. Vectorizar con NumPy nos permite trabajar como se hace en la vida real, procesando matrices completas en milisegundos en vez de poner al computador a sufrir con ciclos for muestra por muestra.

1. Pregunta que nos queda abierta: ¿Qué pasaría si en un año de invierno extremo la lluvia se pasa de 280 mm y los cultivos se inundan? Ahí la relación ya no sería una línea recta sino una curva donde tanto la sequía como la inundación dañan la cosecha, y una sola neurona lineal ya no nos alcanzaría; ¿ahí ya nos tocaría meter una red con capas ocultas?
2.
3. **Salida Final en Consola tras superar los 7 Retos:**

Pregunta que nos queda abierta:

Anexo: Código Completo y Salida Final

Pegar aquí el contenido completo de `perceptron_reto.py` resuelto (fuente de ancho fijo, por ejemplo Consolas 9) y la salida final de la consola.

```
Datos: X tiene forma (120, 2) | y tiene forma (120,)

[OK] RETO 1 (vectorización) superado
[OK] RETO 2 (función sigmoide) superado
[OK] RETO 3 (función de pérdida) superado
[OK] RETO 4A (gradientes) superado
[OK] RETO 4B (actualización de pesos) superado
Época   1 | Pérdida: 0.6931 | Exactitud: 54.17% | w=[0.028, -0.025] b=0.004
Época 100 | Pérdida: 0.3183 | Exactitud: 90.83% | w=[1.333, -1.180] b=0.181
Época 200 | Pérdida: 0.2675 | Exactitud: 90.83% | w=[1.849, -1.651] b=0.237
Época 300 | Pérdida: 0.2467 | Exactitud: 90.83% | w=[2.180, -1.957] b=0.270
Época 400 | Pérdida: 0.2353 | Exactitud: 90.00% | w=[2.426, -2.184] b=0.295
Época 500 | Pérdida: 0.2281 | Exactitud: 90.00% | w=[2.622, -2.366] b=0.315

RESULTADO: pérdida final 0.2281 | exactitud 90.00%
Pesos finales: w1=2.6217, w2=-2.3658, b=0.3152

[OK] RETO 5 (preparación de las entradas) superado
Gráfico guardado: curva_aprendizaje.png
[OK] RETO 6 (frontera de decisión) superado
Gráfico guardado: frontera_decision.png
[OK] RETO 7 (predicción de lotes nuevos) superado
Lluvia 220.0 mm | Temp 23.5 °C -> 99.48% -> EXITOSA
Lluvia  60.0 mm | Temp 30.0 °C -> 0.11% -> NO EXITOSA
Lluvia 150.0 mm | Temp 26.0 °C -> 55.65% -> EXITOSA
```

```
*** ¡Todos los retos superados! Perceptrón completo construido desde cero. ***
```

Código Fuente Resuelto (src/perceptron_reto.py):

```
"""
perceptron_reto.py  –  Actividad grupal Clase 9
Construcción completa del perceptrón con descenso del gradiente.

¿CÓMO FUNCIONA ESTE ARCHIVO?
  - El código tiene 7 RETOS marcados con  >>> RETO X <<<.
  - Cada reto tiene un PUNTO DE CONTROL que verifica si quedó bien resuelto.
  - Al ejecutar el script, se detiene en el primer reto pendiente y muestra una pista.
  - Resuelva el reto, guarde (Ctrl + S) y vuelva a ejecutar:  python perceptron_reto.py
  - Cuando todos los retos estén superados, el script genera los gráficos finales.

Genera: curva_aprendizaje.png, frontera_decision.png
"""

import sys

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


def punto_de_control(superado, reto, pista):
    """Verifica un reto. Si no está superado, muestra la pista y detiene el programa."""
    if superado:
        print(f"[OK] {reto} superado")
    else:
        print(f"\n[PENDIENTE] {reto}")
        print(f"   Pista: {pista}")
        print("   Complete el reto, guarde con Ctrl + S y vuelva a ejecutar el script.")
        sys.exit()


# ============================================================
# 1. Cargar los datos
# ============================================================
df = pd.read_csv("../data/cultivos_cartago.csv")
X = df[["precipitacion_mm", "temperatura_c"]].to_numpy()
y = df["cosecha_exitosa"].to_numpy()
print(f"Datos: X tiene forma {X.shape} | y tiene forma {y.shape}\n")


# ============================================================
# >>> RETO 1 <<<  Vectorización: todas las sumas ponderadas a la vez
# ============================================================
def suma_ponderada(X, w, b):
    """Debe devolver z = w1*x1 + w2*x2 + b para TODAS las filas de X, sin usar ciclos."""
    return np.dot(X, w) + b


X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
z_and = suma_ponderada(X_and, np.array([0.5, 0.5]), -0.7)
punto_de_control(
    z_and is not None and np.allclose(z_and, [-0.7, -0.2, -0.2, 0.3]),
    "RETO 1 (vectorización)",
    "Con la neurona AND de la Clase 8 (w=[0.5, 0.5], b=-0.7) se esperaba [-0.7 -0.2 -0.2 0.3]. "
    "¿Qué operador de NumPy multiplica una matriz por un vector? (Clase 4)",
)
```

```
# ============================================================
# >>> RETO 2 <<<  Una activación que entregue probabilidades
# ============================================================
def sigmoide(z):
    """Debe convertir cualquier número z en un valor entre 0 y 1 (0.5 cuando z = 0)."""
    return 1.0 / (1.0 + np.exp(-z))


prueba = sigmoide(np.array([-6.0, 0.0, 6.0]))
punto_de_control(
    prueba is not None and np.allclose(prueba, [0.0025, 0.5, 0.9975], atol=1e-3),
    "RETO 2 (función sigmoide)",
    "Para z = -6, 0 y 6 se esperaba aproximadamente 0.0025, 0.5 y 0.9975. "
    "Busque la fórmula de la función sigmoide; en NumPy, e elevado a la x es np.exp(x).",
)


def predecir_probabilidad(X, w, b):
    return sigmoide(suma_ponderada(X, w, b))


# ============================================================
# >>> RETO 3 <<<  Medir qué tan mal predice el modelo
# ============================================================
def calcular_perdida(y_real, y_prob):
    """Pérdida logarítmica (entropía cruzada binaria). Debe devolver UN solo número."""
    epsilon = 1e-9
    y_prob = np.clip(y_prob, epsilon, 1 - epsilon)   # evita log(0)
    return -np.mean(y_real * np.log(y_prob) + (1.0 - y_real) * np.log(1.0 - y_prob))


perdida_inicial = calcular_perdida(np.array([1, 0, 1, 0]), np.array([0.5, 0.5, 0.5, 0.5]))
perdida_segura = calcular_perdida(np.array([1]), np.array([0.9]))
punto_de_control(
    perdida_inicial is not None and np.isclose(perdida_inicial, 0.6931, atol=1e-3)
    and np.isclose(perdida_segura, 0.1054, atol=1e-3),
    "RETO 3 (función de pérdida)",
    "Si el modelo dice 0.5 para todo, la pérdida debe ser 0.6931; si acierta con 0.9, debe ser 0.1054. "
    "Busque 'binary cross entropy'. Use np.log y np.mean.",
)


def calcular_exactitud(y_real, y_prob):
    return np.mean((y_prob >= 0.5).astype(int) == y_real)


# ============================================================
# >>> RETO 4 (parte A) <<<  ¿Hacia dónde mover los pesos?
# ============================================================
def calcular_gradientes(X, y_real, y_prob):
    """Debe devolver (gradiente_w, gradiente_b) promediando sobre todos los ejemplos."""
    n = len(y_real)
    error = y_prob - y_real
    gradiente_w = np.dot(X.T, error) / n
    gradiente_b = np.mean(error)
    return gradiente_w, gradiente_b


gw, gb = calcular_gradientes(np.array([[1.0, 0.0], [0.0, 1.0]]), np.array([1, 0]), np.array([0.5, 0.5]))
punto_de_control(
    gw is not None and np.allclose(gw, [-0.25, 0.25]) and np.isclose(gb, 0.0),
    "RETO 4A (gradientes)",
    "Con X=[[1,0],[0,1]], y=[1,0] y y_prob=[0.5,0.5] se esperaba gradiente_w=[-0.25 0.25] y gradiente_b=0.
```

```
"
    "El error es (y_prob - y_real); el gradiente de w usa la transpuesta de X y se divide por n.",
)


# ============================================================
# >>> RETO 5 <<<  Preparar las entradas (resuélvalo cuando el punto de control lo pida)
# ============================================================
media = X.mean(axis=0)
desviacion = X.std(axis=0)
X_esc = (X - media) / desviacion


# ============================================================
# 2. Entrenamiento con descenso del gradiente
# ============================================================
w = np.zeros(2)
b = 0.0
tasa_aprendizaje = 0.1
epocas = 500
historial_perdida = []
historial_exactitud = []

for epoca in range(1, epocas + 1):
    y_prob = predecir_probabilidad(X_esc, w, b)
    gradiente_w, gradiente_b = calcular_gradientes(X_esc, y, y_prob)

    # >>> RETO 4 (parte B) <<<  Dar el paso de aprendizaje
    w = w - tasa_aprendizaje * gradiente_w
    b = b - tasa_aprendizaje * gradiente_b

    if epoca == 1:
        punto_de_control(
            np.allclose(w, -tasa_aprendizaje * gradiente_w) and np.isclose(b, -tasa_aprendizaje *
gradiente_b),
            "RETO 4B (actualización de pesos)",
            "Los pesos no cambiaron o se movieron en la dirección equivocada. Recuerde la Clase 8: el nuevo
peso "
            "depende del peso anterior, de la tasa de aprendizaje y del gradiente. ¿Se suma o se resta para
ir 'cuesta abajo'?",
        )

    historial_perdida.append(calcular_perdida(y, y_prob))
    historial_exactitud.append(calcular_exactitud(y, y_prob))
    if epoca == 1 or epoca % 100 == 0:
        print(f"Época {epoca:>3} | Pérdida: {historial_perdida[-1]:.4f} | "
              f"Exactitud: {historial_exactitud[-1]:.2%} | w=[{w[0]:.3f}, {w[1]:.3f}] b={b:.3f}")


y_prob_final = predecir_probabilidad(X_esc, w, b)
exactitud_final = calcular_exactitud(y, y_prob_final)
print(f"\nRESULTADO: pérdida final {calcular_perdida(y, y_prob_final):.4f} | exactitud
{exactitud_final:.2%}")
print(f"Pesos finales: w1={w[0]:.4f}, w2={w[1]:.4f}, b={b:.4f}\n")

punto_de_control(
    media is not None and desviacion is not None
    and np.allclose(X_esc.mean(axis=0), 0, atol=1e-6) and np.allclose(X_esc.std(axis=0), 1, atol=1e-6),
    "RETO 5 (preparación de las entradas)",
    "El modelo entrena, pero ¿los resultados son buenos? ¿Apareció alguna advertencia (RuntimeWarning)? "
    "Compare los valores de la lluvia (40 a 260) con los de la temperatura (21 a 31). "
    "Recuerde lo que se hizo en la Clase 7 antes de entregar datos a un modelo. "
    "Guarde también en 'media' y 'desviacion' los valores usados: se necesitan en los retos 6 y 7.",
)
```

```
# ============================================================
# 3. Curva de aprendizaje
# ============================================================
fig, (eje1, eje2) = plt.subplots(1, 2, figsize=(12, 4.5))
eje1.plot(historial_perdida, color="red")
eje1.set_title("Pérdida por época")
eje1.set_xlabel("Época")
eje1.grid(True, alpha=0.3)
eje2.plot(historial_exactitud, color="green")
eje2.set_title("Exactitud por época")
eje2.set_xlabel("Época")
eje2.set_ylim(0, 1.05)
eje2.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("curva_aprendizaje.png", dpi=150)
plt.close()
print("Gráfico guardado: curva_aprendizaje.png")

# ============================================================
# >>> RETO 6 <<<  La frontera de decisión
# ============================================================
x1_original = np.linspace(X[:, 0].min(), X[:, 0].max(), 100)
x1_esc = (x1_original - media[0]) / desviacion[0]
# La frontera es la línea donde la neurona duda totalmente (probabilidad 0.5): w1*x1 + w2*x2 + b = 0
x2_esc = -(w[0] * x1_esc + b) / w[1]

punto_de_control(
    x2_esc is not None and np.allclose(w[0] * x1_esc + w[1] * x2_esc + b, 0, atol=1e-6),
    "RETO 6 (frontera de decisión)",
    "¿Para qué valor de z la sigmoide da exactamente 0.5? Escriba esa igualdad con w1, x1, w2, x2 y b, "
    "y despeje x2.",
)
x2_original = x2_esc * desviacion[1] + media[1]

plt.figure(figsize=(9, 6))
plt.scatter(X[y == 1, 0], X[y == 1, 1], color="green", edgecolors="black", label="Exitosa (1)")
plt.scatter(X[y == 0, 0], X[y == 0, 1], color="orange", marker="s", edgecolors="black", label="No exitosa
(0)")
plt.plot(x1_original, x2_original, color="blue", linewidth=2.5, label="Frontera de decisión")
plt.ylim(X[:, 1].min() - 0.5, X[:, 1].max() + 0.5)
plt.title("Frontera de decisión – Cultivos de Cartago")
plt.xlabel("Precipitación mensual (mm)")
plt.ylabel("Temperatura promedio (°C)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("frontera_decision.png", dpi=150)
plt.close()
print("Gráfico guardado: frontera_decision.png")

# ============================================================
# >>> RETO 7 <<<  Usar el modelo con lotes nuevos
# ============================================================
lotes_nuevos = np.array([
    [220.0, 23.5],
    [60.0, 30.0],
    [150.0, 26.0],
])
lotes_nuevos_esc = (lotes_nuevos - media) / desviacion

punto_de_control(
    lotes_nuevos_esc is not None and np.allclose(lotes_nuevos_esc, (lotes_nuevos - media) / desviacion),
    "RETO 7 (predicción de lotes nuevos)",
```

```
    "Los lotes nuevos deben pasar por la MISMA preparación que los datos de entrenamiento. "
    "¿Con qué media y qué desviación? ¿Las de los 3 lotes nuevos o las de los 120 lotes?",
)

for (lluvia, temperatura), probabilidad in zip(lotes_nuevos, predecir_probabilidad(lotes_nuevos_esc, w,
b)):
    decision = "EXITOSA" if probabilidad >= 0.5 else "NO EXITOSA"
    print(f"Lluvia {lluvia:>5.1f} mm | Temp {temperatura:>4.1f} °C -> {probabilidad:.2%} -> {decision}")

print("\n*** ¡Todos los retos superados! Perceptrón completo construido desde cero. ***")
```